SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?

Tingxu Han *
Nanjing University
Mohamed bin Zayed University of Artificial Intelligence
txhan@smail.nju.edu.cn

Yi Zhang
South China University of Technology
202330580551@mail.scut.edu.cn

Wei Song
The University of New South Wales
wei.song1@unsw.edu.au

Chunrong Fang ‡
Nanjing University
fangchunrong@nju.edu.cn

Zhenyu Chen
Nanjing University
zychen@nju.edu.cn

Youcheng Sun
Mohamed bin Zayed University of Artificial Intelligence
youcheng.sun@mbzuai.ac.ae

Lijie Hu ‡
Mohamed bin Zayed University of Artificial Intelligence
lijie.hu@mbzuai.ac.ae

Abstract

Agent skills, structured procedural knowledge packages injected at inference time, are increasingly used to augment LLM agents on software engineering tasks. However, their real utility in end-to-end development settings remains unclear. We present **SWE-Skills-Bench**, the first requirement-driven benchmark that isolates the marginal utility of agent skills in real-world software engineering (SWE). It pairs 49 public SWE skills with authentic GitHub repositories pinned at fixed commits and requirement documents with explicit acceptance criteria, yielding approximately 565 task instances across six SWE subdomains. We introduce a deterministic verification framework that maps each task's acceptance criteria to execution-based tests, enabling controlled paired evaluation with and without the skill. Our results show that skill injection benefits are far more limited than rapid adoption suggests: 39 of 49 skills yield zero pass-rate improvement, and the average gain is only $+1.2\%$. Token overhead varies from modest savings to a 451% increase while pass rates remain unchanged. Only seven specialized skills produce meaningful gains (up to $+30\%$), while three degrade performance (up to -10%) due to version-mismatched guidance conflicting with project context. These findings suggest that agent skills are a narrow intervention whose utility depends strongly on domain fit, abstraction level, and contextual compatibility. SWE-Skills-Bench provides a testbed for evaluating the design, selection, and deployment of skills in software engineering agents. SWE-Skills-Bench is available at https://github.com/GeniusHTX/SWE-Skills-Bench.

*Work done during a research visit at MBZUAI.
‡Corresponding Author.
Pre-print with preliminary results, work in progress.

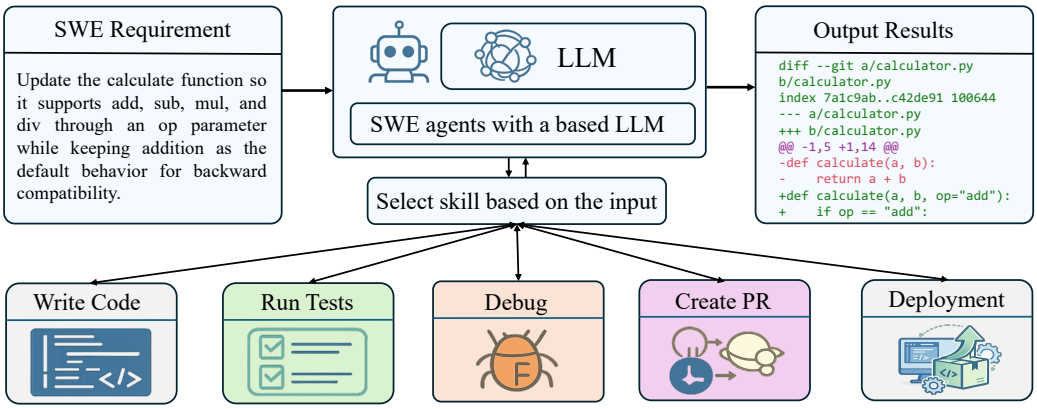


Figure 1: Illustration of how agent skills are used in a software engineering workflow. Given a natural-language requirement, the LLM-based agent selects the most relevant skill from its skill library, including skills such as writing code, running tests, debugging, creating pull requests, and deploying, and injects it into the context window. The agent then executes a series of SWE actions to produce the final software artifacts (such as code) that fulfill the requirement.

1 Introduction

LLM-based agents have been increasingly deployed across a wide range of software engineering (SWE) tasks, from automated code generation and bug fixing [1] to CI/CD pipeline configuration and infrastructure management [2, 3]. Agent Skills are structured markdown packages that encode procedural knowledge,standard operating procedures, code templates, and domain conventions,for consumption by LLM-based agents [4, 5, 6, 7, 8]. At inference time, a skill is simply injected into the agent's context window as a reference document. Unlike fine-tuning or retrieval-augmented generation, no model modification or external retrieval pipeline is required (Figure 1 illustrates how agent skills work given a software engineering task). The ecosystem has grown explosively: over 84,192 skills were created in just 136 days [9].

Despite this rapid adoption, no existing benchmark evaluates SWE skills in real-world software development scenarios. TerminalBench [10] evaluates CLI tasks in multi-file repositories, but does not include a skill-augmentation condition. HumanEval [11] and BigCodeBench [12] target self-contained function completion without multi-file context or skill augmentation. SkillsBench [9] is the first cross-domain benchmark to evaluate agent skills as first-class artifacts under paired skill conditions and deterministic verification. However, it is not specifically designed for software engineering: SWE constitutes only 16 of its 84 tasks, and its primary goal is to measure broad cross-domain skill efficacy rather than requirement satisfaction in real-world development workflows.

A principled benchmark for SWE skill utility must answer a deceptively simple question: *Does the skill help the agent satisfy the task's requirements?* Software engineering is inherently requirement-driven [13, 14, 15]: a task succeeds when every acceptance criterion stated in its specification is met, and unit tests serve as the executable encoding of those criteria. We therefore adopt a **requirement-driven evaluation** methodology: each task is anchored to a requirement document that defines scope and acceptance criteria, and deterministic verifiers based on unit tests are systematically derived from those criteria, establishing full traceability from requirements to test verdicts.

Building on this methodology, we present **SWE-Skills-Bench**, a benchmark designed to isolate the marginal utility of agent skills for software engineering. We curate 49 SWE skills from public repositories, pair each with an authentic GitHub project pinned at a fixed commit, and evaluate under controlled with-skill vs. without-skill conditions. All task instances are verified by deterministic, execution-based checks with no reliance on LLM-as-judge evaluation.

Our main contributions are as follows:

- **Benchmark.** We build SWE-Skills-Bench, a benchmark of 49 real-world SWE skills with $\sim$11 task instances per skill ($\sim$565 total). Tasks are sourced from public skill repositories and evaluated on fixed-commit GitHub projects in containerized environments.

Table 1: Comparison of SWE-Skills-Bench with existing benchmarks. "Skill Cond." indicates whether the benchmark includes agent skills. "Det. Verifier" indicates whether deterministic (non-LLM) verification is included. "SWE-Focused" indicates whether the benchmark is specifically designed for software engineering tasks.

Benchmark	Size	Skill Cond.	Real Projects	Det. Verifier	SWE-Focused
SWE-Bench Verified [1]	500	None	Yes	Yes	Yes
TerminalBench [10]	200	None	Yes	Yes	Yes
HumanEval [11]	164	None	No	Partial	No
SkillsBench [9]	84	Yes	Yes	Yes	Partial
SWE-Skills-Bench	**565**	**Yes**	**Yes**	**Yes**	**Yes**

- **Requirement-driven test harness.** We design an automated unit-testing mechanism that translates each SWE requirement into executable test cases, deterministically verifying whether the specified requirement is fulfilled under both with-skill and without-skill conditions.
- **Empirical findings.** ① Skill injection yields limited marginal gains: 39 of 49 skills produce $\Delta_P = 0$, and the average pass-rate improvement is a modest $+1.2\%$. ② Token overhead is decoupled from correctness: even among skills with zero delta, the token overhead ratio ρ ranges from -78% to $+451\%$, indicating that skills reshape the agent's reasoning path without necessarily improving outcomes. ③ A small subset of 7 skills encoding specialized procedural knowledge—financial risk formulas, cloud-native traffic management, and GitLab CI patterns—delivers meaningful gains up to $+30\%$. ④ Three skills produce negative deltas (up to -10%) when their version-specific conventions conflict with the target project's framework, demonstrating that skill injection carries a structural risk of context interference. These results establish that SWE skill utility is highly domain-specific and context-dependent, favoring targeted skill design over blanket adoption.

2 Related Benchmarks & Datasets

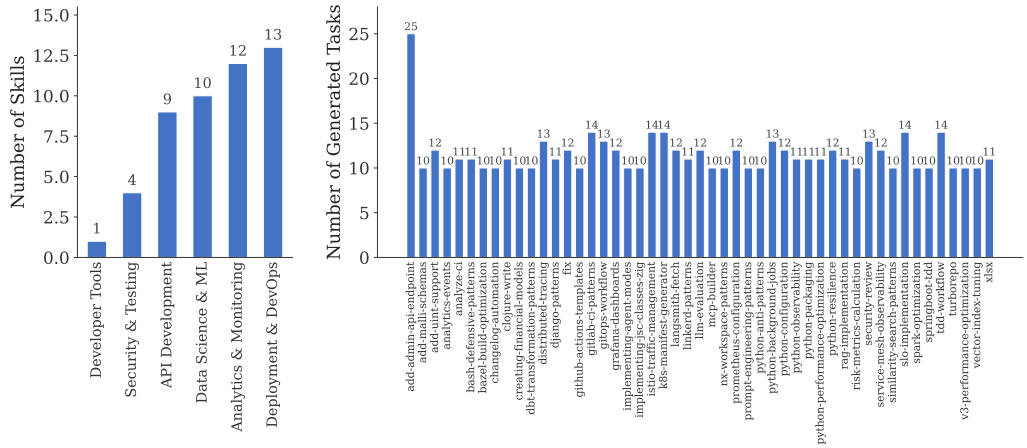
We organize related work into two threads: SWE- and Skill-related benchmarks. Generally, SWE-related benchmarks does not include skills in their evaluation, Skill-related benchmarks does focus on SWE tasks. To the best of our knowledge, we are the first benchmark to evaluate agent skills in software engineering. Table 1 summarizes the key differences.

SWE-related Benchmarks. This line of work can be further divided into SWE real-world benchmarks and code generation benchmarks. SWE real-world benchmarks focus on realistic, project-level software engineering tasks with execution-based verification. SWE-Bench Verified [1] is a human-validated subset of 500 instances from SWE-Bench, drawn from 12 Python repositories and evaluated via fail-to-pass tests. TerminalBench [10] evaluates agents on 200 realistic CLI tasks in containerized environments and provides methodological inspiration for our evaluation setup. However, these benchmarks do not isolate the marginal benefit of injecting procedural skill documents. Code generation benchmarks, in contrast, mainly evaluate models on self-contained coding problems (often algorithmic or snippet-level) without full project context. HumanEval [11] comprises 164 hand-crafted programming challenges at the function level, and therefore does not capture multi-file reasoning, dependency management, or end-to-end SWE workflows.

Skills Benchmarks. SkillsBench [9] takes an important first step toward benchmarking skills as first-class artifacts by comparing agent performance across different skill conditions. Nevertheless, it is not SWE-specific: software engineering forms only a limited subset of its task suite, and the benchmark is not designed around the central success criterion in real-world development—whether explicit requirements are satisfied in repository-grounded workflows. Our work addresses this gap by constructing a requirement-driven benchmark focused exclusively on SWE, where each skill is paired with fixed-commit repositories, explicit requirements, and deterministic execution-based verification.

3 SWE-Skills-Bench Construction

Constructing SWE-Skills-Bench requires answering three key questions in sequence: *which* skills to benchmark, *how* to pair each skill with authentic task instances, and *how* to verify that the



(a) Curated skills across SWE domains **(b) Generated tasks across skills**

Figure 2: The distribution of the curated skills and generated tasks.

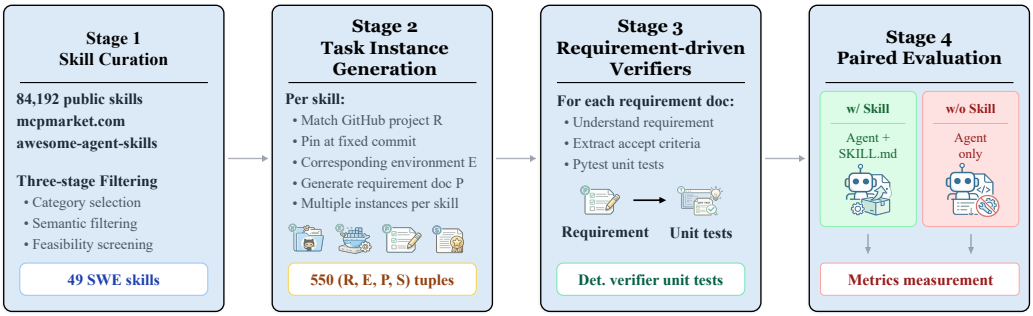


Figure 3: Overview of the SWE-Skills-Bench construction pipeline. We begin with 84,192 public skills and narrow them down through three filtering stages: category selection, semantic filtering, and feasibility screening. This process yields 49 SWE skills (Stage 1). Next, for each skill, we identify a matching GitHub project and generate 565 task instances of the form (R, E, P, S) (Stage 2). For each criterion in the requirements document P, we build deterministic verifiers using pytest unit tests (Stage 3). Finally, we run a paired evaluation that compares agent performance with and without the SKILL.md file, allowing us to measure the effectiveness of the skill (Stage 4).

stated requirements are fulfilled. Our pipeline proceeds in four stages (Figure 3): (1) curating a representative set of SWE skills from large public repositories, (2) generating task instances by pairing each skill with a fixed-commit GitHub project and a requirement document, (3) designing deterministic verifiers that are traceable to the acceptance criteria in each requirement document.

3.1 Skill Curation

The skill ecosystem is vast (84,192 skills created in 136 days [9]) but highly heterogeneous in quality, scope, and evaluability. We curate a deterministic, unit-testable subset through a three-stage filtering pipeline. First, we scan the `mcpmarket` category leaderboard and select six of the nine core categories that best align with software-engineering workflows and are amenable to unit-test evaluation: Developer Tools, Security & Testing, API Development, Data Science & ML, Deployment & DevOps, and Analytics & Monitoring. Second, we apply semantic filtering to exclude generative or subjective skills, retaining only those that target concrete SWE actions such as *fix*, *build*, and *develop*. Third, we exclude candidates whose associated repositories are prohibitively large or incur high environment and setup costs. This pipeline yields 49 skills distributed across the six categories: Deployment & DevOps (13), Analytics & Monitoring (12), API Development (10), Data Science & ML (9), Security & Testing (4), and Developer Tools (1). Figure 2(a) illustrates the distribution.

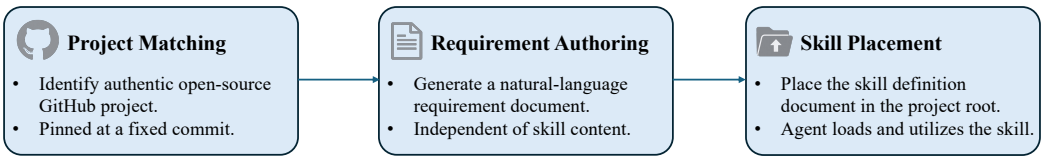


Figure 4: The pipeline of task instance generation.

3.2 Task Instance Generation

As shown in Figure 4, for each curated skill s, we construct approximately 10 task instances following a three-step procedure.

Project matching. We identify an authentic, open-source GitHub project whose technology stack aligns with the skill's domain. The repository is pinned at a fixed commit to ensure reproducibility. Note that we also create a docker container for running each project.

Requirement authoring. Each requirement P is authored to be specific to its target repository and skill-triggering conditions. To maximize structural clarity and eliminate ambiguity, every P adheres to a standardized template comprising: (i) Background, providing the necessary task context; (ii) Requirement, defining the core objective; (iii) File Operations, specifying the files to be modified or created; and (iv) Acceptance Criteria, offering deterministic success metrics. Figure 7 illustrates the prompt utilized to author the requirement and Figure 8 an example of the generated requirement.

Skill placement. During the container preparation phase, the system removes the `.claude/skills` directory from the repository to eliminate interference from pre-existing skills. The activation of skill S is governed by a file-level injection mechanism: the skill document S is copied into the `~/.claude` directory only when the experimental condition requires its use; otherwise, it is omitted. The agent automatically detects and integrates any skills present in this environment. Importantly, the requirement document P never references S, ensuring that the agent's behavior is governed strictly by the physical presence of the skill configuration.

Totally, for each skill, we generate around 10 instances where detailed distributions in Figure 2(b).

3.3 Requirement-driven Verification

The core principle of SWE-Skills-Bench is *requirement-driven verification*. Rather than relying on subjective judgments, we convert every acceptance criterion in the requirement document P into objective, deterministic tests, ensuring that each test outcome is directly traceable to a specific requirement. We provide P (together with repository metadata such as repo path, language, and available test commands) to a fixed "professional test engineer" prompt template, which instructs the model to (i) enumerate testable behaviors from each acceptance criterion, (ii) instantiate representative and edge-case scenarios, and (iii) encode them into a deterministic `pytest` test file with strong discriminative power (i.e., tests must run the produced code and verify concrete outputs/structures rather than keyword-level heuristics). The prompt also enforces structural constraints such as a minimum number of test cases and per-test docstrings. The prompt template is shown in Figure 6.

Concretely, for each instance we create a container from a base image, clone the target repository into the container workspace, and complete environment setup. We then pass the task document (i.e., the requirement document P) through the above prompt template to drive test generation, and use the task document as the prompt to Claude Code for implementation.

3.4 Task Formulation

Each task instance is a tuple (R, E, P, S): a GitHub repository R pinned at a fixed commit and the corresponding containerized running environment, a natural-language requirement document P that specifies tasks, and optionally a skill document S. The agent (claude code specifically) must produce code changes, configuration files, or execution artifacts that satisfy the requirements in P given the code repository R and environment E.

Table 2: Evaluation results across all 49 skills. Pass$^+$ and Pass$^-$ denote pass rates with and without skill injection, respectively. ΔP is the skill utility delta, C^+ and C^- are average token costs, ρ is the token overhead ratio, and CE is cost efficiency. Best viewed in color.

Skills	#Tasks	Pass$^+$	Pass$^-$	ΔP	C^+	C^-	ρ	CE
add-uint-support	12	100.0%	100.0%	0.0%	880K	414K	+112.6%	—
analytics-events	10	100.0%	100.0%	0.0%	321K	157K	+104.6%	—
analyze-ci	11	100.0%	100.0%	0.0%	66K	74K	-10.6%	—
dbt-transformation-patterns	10	100.0%	100.0%	0.0%	422K	208K	+103.2%	—
gitops-workflow	13	100.0%	100.0%	0.0%	130K	57K	+127.1%	—
grafana-dashboards	12	100.0%	100.0%	0.0%	150K	116K	+29.3%	—
implementing-agent-modes	10	100.0%	100.0%	0.0%	342K	655K	-47.8%	—
k8s-manifest-generator	14	100.0%	100.0%	0.0%	98K	51K	+91.2%	—
langsmith-fetch	12	100.0%	100.0%	0.0%	102K	97K	+5.9%	—
llm-evaluation	12	100.0%	100.0%	0.0%	238K	203K	+17.6%	—
mcp-builder	10	100.0%	100.0%	0.0%	273K	200K	+36.1%	—
nx-workspace-patterns	10	100.0%	100.0%	0.0%	417K	365K	+14.5%	—
prometheus-configuration	12	100.0%	100.0%	0.0%	225K	312K	-27.8%	—
python-anti-patterns	10	100.0%	100.0%	0.0%	274K	490K	-44.1%	—
python-background-jobs	13	100.0%	100.0%	0.0%	839K	249K	+236.8%	—
python-observability	11	100.0%	100.0%	0.0%	271K	105K	+157.5%	—
python-packaging	11	100.0%	100.0%	0.0%	167K	74K	+123.9%	—
python-performance-optimization	11	100.0%	100.0%	0.0%	91K	96K	-5.1%	—
python-resilience	12	100.0%	100.0%	0.0%	119K	529K	-77.6%	—
rag-implementation	11	100.0%	100.0%	0.0%	258K	179K	+44.5%	—
service-mesh-observability	12	100.0%	100.0%	0.0%	733K	133K	+450.8%	—
slo-implementation	14	100.0%	100.0%	0.0%	144K	241K	-40.2%	—
spark-optimization	10	100.0%	100.0%	0.0%	223K	180K	+23.9%	—
v3-performance-optimization	10	100.0%	100.0%	0.0%	237K	544K	-56.4%	—
add-admin-api-endpoint	25	84.0%	84.0%	0.0%	243K	232K	+4.4%	—
add-malli-schemas	10	90.0%	90.0%	0.0%	646K	433K	+49.2%	—
bash-defensive-patterns	11	90.9%	90.9%	0.0%	565K	231K	+144.3%	—
bazel-build-optimization	10	90.0%	90.0%	0.0%	316K	790K	-60.0%	—
changelog-automation	10	70.0%	70.0%	0.0%	128K	274K	-53.3%	—
clojure-write	11	81.8%	81.8%	0.0%	579K	869K	-33.4%	—
creating-financial-models	10	90.0%	90.0%	0.0%	197K	195K	+0.7%	—
fix	12	91.7%	91.7%	0.0%	202K	80K	+153.0%	—
github-actions-templates	10	70.0%	70.0%	0.0%	85K	61K	+39.1%	—
implementing-jsc-classes-zig	10	90.0%	90.0%	0.0%	1.1M	940K	+22.0%	—
python-configuration	12	91.7%	91.7%	0.0%	199K	154K	+29.7%	—
security-review	13	92.3%	92.3%	0.0%	301K	299K	+0.9%	—
turborepo	10	50.0%	50.0%	0.0%	753K	262K	+187.9%	—
vector-index-tuning	10	90.0%	90.0%	0.0%	475K	400K	+18.8%	—
xlsx	11	36.4%	36.4%	0.0%	1.5M	1.8M	-18.1%	—
risk-metrics-calculation	10	100.0%	70.0%	+30.0%	507K	778K	-34.8%	-0.86
gitlab-ci-patterns	14	78.6%	64.3%	+14.3%	326K	205K	+58.6%	0.24
prompt-engineering-patterns	10	100.0%	90.0%	+10.0%	218K	149K	+46.4%	0.22
similarity-search-patterns	10	100.0%	90.0%	+10.0%	144K	213K	-32.4%	-0.31
distributed-tracing	13	100.0%	92.3%	+7.7%	115K	165K	-30.4%	-0.25
tdd-workflow	14	28.6%	21.4%	+7.1%	148K	83K	+78.6%	0.09
istio-traffic-management	14	100.0%	92.9%	+7.1%	95K	121K	-22.0%	-0.32
springboot-tdd	10	70.0%	80.0%	-10.0%	236K	374K	-36.8%	0.27
linkerd-patterns	11	90.9%	100.0%	-9.1%	248K	165K	+50.3%	-0.18
django-patterns	11	81.8%	90.9%	-9.1%	482K	462K	+4.2%	-2.16
Average	565	91.0%	89.8%	+1.2%	335K	303K	+10.5%	—

In our evaluation methodology, every acceptance criterion in the requirement document P is mapped to deterministic verifier, establishing full traceability from requirements to test verdicts.

4 Results of SWE-Skills-Bench

4.1 Experimental Setup

All experiments run in Docker containers (Ubuntu 24.04, CPU-only) with per-task resource limits specified in the task configuration. The agent is Claude Code [16] with the Claude Haiku 4.5. For each task, we evaluate it under use-skill or no-skill conditions. In the use-skill condition, `SKILL.md`

is placed in the project root directory. The agent discovers and applies it autonomously without explicit instruction.

4.2 Evaluation Metrics

Let $\mathcal{T}_s = \{t_1, \ldots, t_N\}$ denote the set of N task instances associated with skill s. For each instance t_i, let $v_i^+ \in \{0, 1\}$ and $v_i^- \in \{0, 1\}$ be the binary pass/fail verdicts under the with-skill and without-skill conditions, respectively, and let c_i^+ and c_i^- be the corresponding token costs (total input and output tokens consumed by the agent).

- **Pass Rate.** The primary metric. For each condition:

$$\text{Pass}^+(s) = \frac{1}{N}\sum_{i=1}^{N} v_i^+, \qquad \text{Pass}^-(s) = \frac{1}{N}\sum_{i=1}^{N} v_i^- \tag{1}$$

- **Skill Utility Delta** (Δ). Measures the marginal benefit of skill injection:

$$\Delta P(s) = \text{Pass}^+(s) - \text{Pass}^-(s) \tag{2}$$

  Positive Δ indicates the skill helps, zero indicates irrelevance, and negative Δ indicates interference.

- **Token Cost.** The average token consumption per condition (with $(+)$ or without $(-)$ skills):

$$C^+(s) = \frac{1}{N}\sum_{i=1}^{N} c_i^+, \qquad C^-(s) = \frac{1}{N}\sum_{i=1}^{N} c_i^- \tag{3}$$

  and the token overhead ratio induced by skill injection:

$$\rho(s) = \frac{C^+(s) - C^-(s)}{C^-(s)} \tag{4}$$

  A positive ρ indicates that the skill increases token consumption; comparing ρ with Δ reveals whether skill-induced gains justify their inference cost.

- **Cost Efficiency.** To jointly assess performance gains and token overhead, we define the cost efficiency of a skill as:

$$\text{CE}(s) = \frac{\Delta P(s)}{\rho(s)}, \tag{5}$$

  Intuitively, $\text{CE}(s)$ quantifies the success-rate improvement obtained per unit of relative token increase. Larger positive values indicate greater performance gains per token cost, whereas negative values indicate that the skill either degrades performance or incurs disproportionate overhead.

4.3 Evaluation Results

Table 2 presents the full evaluation results across all 49 skills. At the aggregate level, skill injection raises the average pass rate by a modest $+1.2\%$ (from 89.8% to 91.0%) while increasing average token consumption by 10.5%. Beneath these averages, however, the per-skill behavior is highly heterogeneous. We structure our analysis around five key findings that show when skills help, when they are redundant, and when they actively disrupt the agent's reasoning.

Finding 1: Skill injection yields limited marginal gains on pass rate. For the 49 evaluated skills, 39 (roughly 80%) produce $\Delta P = 0$, meaning that skill injection neither helps nor hurts the agent's task-level success rate. Among these, 24 skills achieve $\text{Pass}^+ = \text{Pass}^- = 100\%$, indicating that the base model already possesses sufficient capability to solve every task instance without any skill guidance. The remaining 15 skills share identical but imperfect pass rates across conditions (e.g., `xlsx` at 36.4%, `turborepo` at 50.0%). This suggests that the bottleneck lies not in the absence of domain knowledge, which the skill ostensibly provides, but in deeper capability gaps such as complex multi-step reasoning, unfamiliar API surfaces, or brittle evaluation harnesses. For these skills, improving pass rates likely requires either fundamentally rethinking the skill content, upgrading the base model, or relaxing evaluation criteria, rather than simply injecting more contextual guidance. Overall, in software engineering, the average skill utility delta is $+1.2\%$, confirming that skill

injection is not a universal performance booster but rather a targeted intervention whose benefits are concentrated in a small subset of skills.

Finding 2: Token overhead is decoupled from performance gains. Even when $\Delta P = 0$, skills can still have a large impact on inference cost. Within the 24 skills that achieve perfect pass rates in both conditions, the token overhead ratio ρ ranges from -77.6% (`python-resilience`) to $+450.8\%$ (`service-mesh-observability`). This spread shows that injecting a skill can change the agent's reasoning path without changing the final result. In some cases, it makes the reasoning more efficient, while in others, it lengthens the process with redundant exploration. Of the 24 skills with perfect scores in both conditions, 8 use fewer tokens when the skill is injected ($\rho < -5\%$). The savings are sometimes large, reaching 77.6% for `python-resilience` and 56.4% for `v3-performance-optimization`. This suggests that these skills guide the agent toward a more direct solution path. But more generally, the other 16 skills use more tokens under skill injection ($\rho > +5\%$), often by a wide margin. For example, `service-mesh-observability` incurs a 450.8% overhead, and `python-background-jobs` incurs a 236.8% overhead. Crucially, ρ and ΔP exhibit no consistent correlation across the full set of 49 skills: several skills with $\Delta P > 0$ simultaneously reduce token consumption (e.g., `risk-metrics-calculation` with $\rho = -34.8\%$), while many $\Delta P = 0$ skills dramatically increase it. This decoupling implies that the mechanisms by which skills affect reasoning efficiency are largely independent of those that affect correctness.

Finding 3: A small subset of skills delivers meaningful improvements. Seven skills achieve $\Delta P > 0$, with gains ranging from $+7.1\%$ to $+30.0\%$. The most effective skill, `risk-metrics-calculation` ($\Delta P = +30.0\%$, $\rho = -34.8\%$), simultaneously improves correctness and reduces token cost, representing the ideal outcome of skill injection. At the other end, `tdd-workflow` yields a modest $+7.1\%$ improvement at the expense of a 78.6% token overhead, resulting in low cost efficiency ($\mathrm{CE} = 0.09$). In this scenario, the agent achieves better performance at the cost of using many more tokens. This is because the skill functions as a checklist. It forces the agent to attend to edge case deliverables that are often overlooked in the no-skill setting. This added structure can improve correctness by making the agent more likely to cover required but easily missed steps. However, this added coverage also requires more verification and follow-through, so the gains often come with higher token costs.

Finding 4: Skills can actively degrade performance through context interference. Three skills exhibit negative ΔP: `springboot-tdd` (-10.0%), `linkerd-patterns` (-9.1%), and `django-patterns` (-9.1%). These regressions point to a structural risk inherent in the skill injection mechanism: the mismatch between the *holistic* scope of a skill and the *focused* requirements of individual tasks. Each skill is authored as a comprehensive reference for its technical domain, encoding best practices that span architecture, coding conventions, testing strategies, and error handling. When a task exercises only a narrow slice of this knowledge, the surplus context can interfere with the agent's reasoning in several ways. First, the rich set of patterns and strategies described in the skill unnecessarily expands the agent's decision space, prompting deliberation over design choices the task does not warrant. Second, production-grade templates may steer the agent toward over-fitted solutions that rigidly follow the skill's examples rather than adapting to the task's actual requirements. Third, the skill text itself competes for the finite context window, displacing tokens that would otherwise be devoted to understanding the task description and the codebase.

The `linkerd-patterns` case illustrates this mechanism as shown in Figure 5. The task asks the agent to produce a `Server` CRD and a `ServerAuthorization` CRD that enforce mTLS identity verification for a gRPC service. The skill packages seven templates covering the full Linkerd stack, installation, namespace injection, service profiles, traffic splitting, server authorization, HTTPRoute, and multi-cluster setup. Among them, Template 5 demonstrates exactly the two CRDs the task requires, but with different concrete values: it uses API version `v1beta1` with `proxyProtocol: HTTP/1`, and shows multiple authorization modes including both `meshTLS` and `unauthenticated` access with CIDR ranges. This near-match triggers severe context pollution, thereby interfering with the model's understanding of the task.

Without skill injection, the agent reasons from first principles and produces a correct solution: it selects `v1beta3` for the `Server`, sets `proxyProtocol: gRPC` to match the application, and configures `ServerAuthorization` with the standard `client.meshTLS.serviceAccounts` field. With the skill injected, Template 5 anchors the agent and the errors compound through three stages:

Task: Add Linkerd mTLS Verification Example. Create server-policy.yaml containing a **Server** CRD selecting the server pods and a **ServerAuthorization** CRD requiring mTLS identity. Restrict access to only the client's ServiceAccount identity.

Skill Template 5 (Injected)

```
# Template 5: Server
Authorization
apiVersion:
policy.linkerd.io/v1beta1          (1)
kind: Server
spec:
  podSelector:
    matchLabels:
      app: {{ .app }}
  port: {{ .port }}
  proxyProtocol: HTTP/1            (2)
---
    meshTLS:
      serviceAccounts:
      - name: {{ .sa }}
    unauthenticated: true
    networks:                      (3)
    - cidr: 10.0.0.0/8
kind: NetworkPolicy
```

(a) Without Skill

```
# Server CRD
apiVersion: .../v1beta3
kind: Server
metadata:
  name: mtls-server-grpc
spec:
  podSelector:
    matchLabels:
      app: mtls-server
  port: 9090
  proxyProtocol: gRPC

# ServerAuthorization CRD
client:
    meshTLS:
      serviceAccounts:
      - name: mtls-client
```

(b) With Skill

```
# Server CRD
apiVersion: .../v1beta1            (1)
kind: Server
metadata:
  name: server
spec:
  podSelector:
    matchLabels:
      app: server
  port: http
  proxyProtocol: HTTP/1            (2)

# ServerAuthorization CRD
rules:
  - metricsServers:
    - kind: ServiceAccount         (3)
      name: client
# Unrequested resource
kind: NetworkPolicy ...
```

Figure 5: Context interference in the `linkerd-patterns` skill ($\Delta P = -9.1\%$). The task requires a `Server` CRD and a `ServerAuthorization` CRD enforcing mTLS identity verification for a gRPC service. **Left:** Template 5 from the injected skill, which near-matches the task but encodes different concrete values: API version `v1beta1` with `proxyProtocol: HTTP/1`, and multiple authorization modes (meshTLS, unauthenticated, and CIDR-based). **Center:** Without the skill, the agent reasons from first principles and produces a correct solution using `v1beta3`, `gRPC`, and standard `meshTLS.serviceAccounts`. **Right:** With the skill, the agent's output degrades through three stages, each traceable to a specific region of the template (matched by circled numbers): ① Surface anchoring, the agent copies `v1beta1` and `HTTP/1` verbatim; ② Hallucination, while reconciling the template's mixed authorization modes, the agent fabricates a nonexistent `rules/metricsServers` field; ③ Concept bleed, the template's `NetworkPolicy` example causes the agent to append an unrequested resource, conflating Linkerd-level and Kubernetes-level authorization.

1. **Surface anchoring.** The agent copies the template's API version (`v1beta1`) and protocol (`HTTP/1`) verbatim instead of adapting them to the task's gRPC context. The template's concrete values override the agent's own knowledge of the correct configuration.
2. **Hallucination.** While attempting to reconcile the template's authorization pattern with the task's identity-verification requirement, the agent fabricates a nonexistent `rules/metricsServers` field in the `ServerAuthorization` spec—a field that appears in no version of the Linkerd CRD. The cognitive load of processing seven templates simultaneously degrades the agent's ability to distinguish valid API fields from plausible-sounding constructs.
3. **Concept bleed.** The agent appends an unrequested `NetworkPolicy` resource, conflating Template 5's multiple authorization modes (meshTLS identity, unauthenticated access, CIDR-based network rules) with the Kubernetes-native `NetworkPolicy` API. The skill's broad coverage causes concepts from adjacent domains to leak into the solution.

This explains the seemingly paradoxical outcome: a skill containing *objectively relevant* content nonetheless *degrades* performance. The practical implication is that skill design should favor abstract guidance patterns over concrete, opinionated templates with hard-coded parameter values, as the latter risk anchoring the agent on specifics that may not transfer to the target task.

5 Discussion & Future Directions

SWE-Skills-Bench is an ongoing effort toward systematically understanding how procedural skill injection affects LLM-based software engineering agents. The results presented in this paper represent a snapshot of a larger, actively evolving research program. While our current findings already reveal several actionable insights, most notably that skill utility is highly domain-specific and that context interference is a tangible risk, the benchmark in its present form covers only a fraction of the design space. We view this work as laying the foundation and evaluation methodology; substantial extensions along multiple axes are underway and planned.

Multi-model evaluation. All experiments in this work use a single agent configuration: Claude Code with Claude Haiku 4.5. Skill utility, however, is likely modulated by the base model's existing knowledge and reasoning capabilities. A stronger model may already internalize the procedural knowledge encoded in a skill, rendering the skill redundant, while a weaker model may lack the capacity to effectively leverage the injected context. We plan to evaluate SWE-Skills-Bench across a diverse set of foundation models—varying in scale, training data composition, and architecture—to disentangle model-intrinsic capability from skill-induced improvement and to identify which model–skill pairings yield the most favorable cost–performance trade-offs.

Diverse agent scaffolds. Beyond the choice of foundation model, the agent scaffold, i.e., the orchestration framework that governs tool use, planning, and context management, can significantly mediate how a skill is consumed. Different scaffolds may allocate context budgets differently, employ distinct retrieval strategies for long skill documents, or impose varying levels of structure on the agent's reasoning trace. We intend to benchmark skill utility across multiple open-source and proprietary agent frameworks (e.g., SWE-agent, OpenHands, Aider) to assess whether our findings generalize beyond the specific scaffold used in this study.

Skill design principles. Our analysis of context interference (Finding 4) suggests that the form of a skill, not just its content, plays a critical role in determining utility. Skills that rely on concrete, opinionated templates with hard-coded parameter values risk anchoring the agent on specifics that may not transfer to the target task, whereas skills that encode abstract guidance patterns may offer more robust benefits. A promising direction is to study how skill granularity, abstraction level, and structural organization (e.g., modular sections vs. monolithic documents) affect downstream performance, with the goal of deriving empirically grounded guidelines for skill authors.

Dynamic skill selection and composition. The current evaluation framework assumes a one-skill-per-task setting in which the relevant skill is pre-placed in the project. In realistic deployments, agents must select from a large skill library or compose multiple skills at inference time. Evaluating skill retrieval accuracy, multi-skill interaction effects, and the robustness of skill selection under ambiguity constitutes an important extension of our benchmark

References

[1] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. 2, 3

[2] John Yang, Akshara Prabhakar, et al. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. 2

[3] Wei Song, Haonan Zhong, Ziqi Ding, Jingling Xue, and Yuekang Li. Help or hurdle? rethinking model context protocol-augmented large language models. *arXiv preprint arXiv:2508.12566*, 2025. 2

[4] Anthropic. Equipping agents for the real world with agent skills. Anthropic Engineering Blog, 2025. 2025a. 2

[5] Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Mem^{p}: Exploring agent procedural memory. *arXiv preprint arXiv:2508.06433*, 2025. 2

[6] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024. 2

[7] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024. 2

[8] Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*, 2026. 2

[9] Xiang Li, Yang Liu, Wei Chen, et al. SkillsBench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026. 2, 3, 4

[10] Mark A. Merrill et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026. 2, 3

[11] Mark Chen et al. Evaluating large language models trained on code. In *arXiv preprint arXiv:2107.03374*, 2021. 2, 3

[12] Terry Yue Zhuo et al. BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. In *International Conference on Learning Representations (ICLR)*, 2025. 2

[13] Ian Sommerville. *Software Engineering*. Pearson Education, 10th edition, 2015. 2

[14] Pamela Zave. Classification of research efforts in requirements engineering. *ACM Computing Surveys*, 29(4):315–321, 1997. 2

[15] Klaus Pohl. *Requirements Engineering: Fundamentals, Principles, and Techniques*. Springer, Heidelberg, 2010. 2

[16] Anthropic. Claude code: An agentic coding tool. GitHub, 2025. 2025b. 6

System prompt

You are a professional test engineer. Your task is to generate automated test suites that evaluate whether an AI Agent has correctly completed a given programming task.

Context
An AI Agent is given a programming task to complete within a real software environment. After the Agent finishes, the test suite you generate is executed to automatically determine whether the task was completed correctly.
Core Principles
1. **Discriminative Power**: Tests must reliably distinguish between "genuinely completed" and "superficially plausible" outputs.
2. **Behavioral Verification**: Execute the code and verify its actual output — do not rely solely on static checks such as keyword matching or file existence.
3. **Completeness**: Cover all acceptance criteria specified in the task, including boundary conditions and error handling.
4. **Non-Trivial Assertions**: Verify the correctness of values, data structures, and logic — not merely their presence.

Test Design Guidelines
Required Strategies
- **Run and verify**: Execute the Agent's code and check return values, output content, and side effects.
- **Structural validation**: For generated configs, schemas, or data files, parse and validate the structure and semantics — not just syntax.
- **Edge-case testing**: Include tests for boundary inputs, missing fields, and expected error behaviors.
Prohibited Patterns
- Keyword-only assertions (e.g., `assert "keyword" in source_code`)
- Overly permissive checks (e.g., `assert len(results) >= 1`)
- File-existence-only checks without content verification
- Any assertion that a trivially incorrect output could pass
Output Requirements
- Generate ≥ 10 test cases per task, spanning multiple difficulty levels.
- Each test must include a docstring explaining what it verifies.
- Output only executable test code — no explanatory prose.

User Prompt

Please generate a test suite for the following AI Agent task.

Task ID
{task_id}
Environment
- Project root: {project_root}
- Language / Framework: {language}
- Available toolchain: {toolchain}
Task Description
{task_description}
Acceptance Criteria
{acceptance_criteria}
Constraints
- Per-test timeout: {timeout} seconds
- Available tools in the execution environment: {available_tools}

Figure 6: The prompt used for requirement-driven verification generation.

 # Task Requirement Document Generator — Prompt Template

You are a ****task requirement document generator****. Generate a ****Task Requirement Document**** (Markdown) based on the provided {{information_sources}} and {{skill_reference}}. The task requirement should represent a realistic {{task_scope}} within the {{target_context}}.

The generated document must be ****self-sufficient****: a {{task_executor}} reading only this document must understand what to build, where to make changes, and what counts as done.

Core Principles

1. Match the {{target_context}}'s real ****{{tech_constraints}}****.
2. Stay within the ****problem scope**** defined by {{skill_reference}}.
3. Focus on concrete ****objectives, constraints, {{artifact_location}}s, and verifiable outcomes****.
4. ****Do not leak**** {{skill_reference}} methods, best practices, or implementation patterns.

Information Sources

- ****Configuration****: id, name, description, type, evaluation config from {{config_source}}
- ****Skill Reference****: {{{skill_reference}} full content}
- ****Task Template****: {{{task_template}} full content}

Required Sections

Section	Purpose
Background	Context a {{task_executor}} needs
{{artifact_section_name}}	Concrete {{artifact_type}} paths to create/modify
Requirements	Behavior, constraints, edge cases — ****not**** solutions
Acceptance Criteria	Observable, verifiable "done" conditions

Specificity Rules

- Name exact {{domain_specifics}} whenever part of the goal.
- Name exact {{artifact_type}} paths — avoid vague entries like "{{vague_example}}".
- Name important edge cases, validation rules, and failure modes.

Anti-Patterns

- ✗ Rewriting {{skill_reference}} steps/patterns into the task requirement.
- ✗ Guiding language ("recommended to use", "suggest adopting").
- ✗ Vague {{artifact_type}} locations or validation criteria.
- ✗ Technology stack mismatching the {{target_context}}.

Self-Check

- [ ] {{task_executor}} can complete without hidden knowledge.
- [ ] Every {{artifact_type}} entry is concrete and {{path_format}}.
- [ ] Acceptance Criteria are outcome-based.
- [ ] No {{skill_reference}} methodology leaked.

Template Variables

Variable	Description	Example
`{{task_executor}}`	Who executes the task	agent, developer
`{{target_context}}`	Project environment	repository, codebase
`{{skill_reference}}`	Knowledge doc (not to leak)	SKILL.md
`{{tech_constraints}}`	Technical boundaries	tech stack, testing style
`{{artifact_type}}`	Deliverable unit	file, component
`{{artifact_section_name}}`	Section header for deliverables	Files to Create/Modify
`{{path_format}}`	How paths are expressed	repository-relative
`{{domain_specifics}}`	Domain nouns to be precise about	APIs, schemas, events
`{{vague_example}}`	Anti-pattern example	"files under src/"
`{{config_source}}`	Configuration origin	benchmark_config.yaml
`{{task_scope}}`	What the task represents	development task
`{{information_sources}}`	Generator inputs	repo info, eval config

Figure 7: The prompt used for task instance requirement generation.

 # Task: Extend Unsigned Integer Type Coverage in PyTorch Operators
Background
The PyTorch codebase (https://github.com/pytorch/pytorch) has partial support for unsigned integer types across its operator library. Several arithmetic and mathematical operators currently lack support for `uint16`, `uint32`, and `uint64` tensor types. When users pass tensors of these dtypes to unsupported operators, the runtime raises a dispatch error. The type dispatch infrastructure needs to be extended so that additional operators can accept unsigned integer inputs.
Files to Modify
- `aten/src/ATen/native/BinaryOps.cpp` — Dispatch registration for arithmetic ops (add, sub, mul, floor_divide, remainder) and bitwise ops (and, or, xor, lshift, rshift)
- `aten/src/ATen/native/cpu/BinaryOpsKernel.cpp` — CPU kernel implementations for binary operators
- `aten/src/ATen/native/TensorCompare.cpp` — Dispatch registration for comparison ops (eq, ne, lt, le, gt, ge)
- `aten/src/ATen/native/GcdLcm.cpp` — GCD operator dispatch and implementation
Requirements
1. Operators to support
- The implementation MUST explicitly add `uint16`, `uint32`, and `uint64` support for the following operators (operator names correspond to native ATen symbols and Python API where applicable):
- Arithmetic: `add` (aten::add), `sub` (aten::sub), `mul` (aten::mul)
- Integer division / remainder: `floor_divide` (aten::floor_divide), `remainder` (aten::remainder)
- Bitwise and shifts: `bitwise_and` (aten::bitwise_and), `bitwise_or` (aten::bitwise_or), `bitwise_xor` (aten::bitwise_xor), `lshift`/`left_shift` (aten::lshift), `rshift`/`right_shift` (aten::rshift)
- GCD: `gcd` (aten::gcd) when present in the codebase
- Comparisons: `eq`, `ne`, `lt`, `le`, `gt`, `ge` (aten::* comparison ops)
2. Scope of changes
- For each operator above, update both the operator registration/type-dispatch tables and the CPU kernel implementations under `aten/src/ATen/native/` so that the operator accepts unsigned integer tensors without raising dispatch errors.
- If a kernel implementation is missing for an unsigned dtype, add an explicit kernel path (reuse signed-integer implementation where semantics match, or add a thin adapter that performs identical, dtype-preserving logic).
- Do NOT change operators that are inherently floating-point-only (e.g., `sqrt`, `sin`, `exp`).
3. Semantics and dtype rules
- When all inputs are the same unsigned integer dtype, the operator should preserve that dtype for outputs where that is semantically correct (e.g., `add(uint32, uint32) -> uint32`).
- For mixed-type inputs, follow existing PyTorch promotion rules; do not introduce new promotion behavior beyond existing signed-integer promotion rules.
- Integer division behavior: implement `floor_divide` and `remainder` semantics consistent with current PyTorch integer ops (no conversion to floating point), and ensure results for unsigned inputs match the mathematical remainder/quotient for non-negative integers.
4. Compatibility and robustness
- Ensure changes compile and pass existing unit tests unrelated to unsigned support.
- Provide fallbacks or clear error messages for operator combinations that remain unsupported (e.g., mixing unsigned with types that cannot be sensibly combined).
5. Implementation notes for contributors
- Prefer adding dtype coverage via dispatch table entries and small adapters rather than rewriting algorithmic kernels.
- Include small unit tests for each operator (see Acceptance Criteria) to validate behavior on sample inputs.
Expected Functionality
- Operators that previously raised dispatch errors for `uint32` or `uint64` tensors now execute successfully and return correct results
- The output tensor dtype matches the input tensor dtype when all inputs share the same unsigned type
- Operators that only make sense for floating-point types remain unchanged
Additionally, the repository should include minimal unit tests that demonstrate correct behavior for each operator listed in the Requirements (examples in Acceptance Criteria). These tests should validate dtype preservation, numerical correctness for representative values, and reasonable behavior for edge cases (e.g., zero, max-value, boundary shifts).
Acceptance Criteria
- The listed operators accept `uint16`, `uint32`, and `uint64` inputs without raising dispatch errors.
- Arithmetic, division, remainder, bitwise, GCD, and comparison results are numerically correct for representative unsigned inputs.
- When all inputs share the same unsigned dtype, the result keeps that dtype wherever PyTorch's existing promotion rules do not require otherwise.
- Edge cases including zero values, maximum representable values, and boundary shift counts behave consistently and do not crash.
- Floating-point-only operators remain unchanged and unsigned support is limited to operators with well-defined integer semantics.

Figure 8: An example of the generated requirement in SWE-Skills-Bench.